

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

«ПРОЦЕДУРЫ, ФУНКЦИИ, ТРИГГЕРЫ В PostgreSQL»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Журбина Марина Андреевна

Факультет Прикладной информатики

Группа K3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

Цель работы:

Овладеть практическими создания и использования процедур, функций и триггеров в базе данных PostgreSQL.

Практическое задание:

1. Создать 3 процедуры для индивидуальной БД согласно варианту 7 ЛР2:
 - получение расписания занятий группы на определённый день недели;
 - запись (зачисление) слушателя на курс/в группу;
 - получение перечня свободных лекционных аудиторий на заданный день недели (и время).
2. Создать триггеры по индивидуальному заданию (вариант 2.1): реализовать 7 триггеров, обеспечивающих целостность данных и бизнес-правила предметной области «Курсы».

Выполнение:

Наименование БД: Courses.

Вариант: 7.

Скрипт создания процедур и триггеров:

<https://drive.google.com/file/d/1QmhsxbSTm-tbgUGrPvcdToCIXLY9xr-4/view?usp=sharing>

Результат выполнения скрипта:

```

Courses=# \i 'D:/BD/lr5/lr5.sql'
psql:D:/BD/lr5/lr5.sql:1: ЗАМЕЧАНИЕ:  схема "lr3" уже существует, пропускается
CREATE SCHEMA
SET
psql:D:/BD/lr5/lr5.sql:6: ЗАМЕЧАНИЕ:  процедура lr3.pr_get_group_schedule_by_dow(pg_catalog.int4,pg_catalog.int4,refcursor) не существует, пропускается
DROP PROCEDURE
CREATE PROCEDURE
psql:D:/BD/lr5/lr5.sql:43: ЗАМЕЧАНИЕ:  процедура lr3.pr_enroll_trainee_to_group(pg_catalog.int4,pg_catalog.int4) не существует, пропускается
DROP PROCEDURE
CREATE PROCEDURE
psql:D:/BD/lr5/lr5.sql:70: ЗАМЕЧАНИЕ:  процедура lr3.pr_free_lecture_rooms_by_dow(pg_catalog.int4,pg_catalog.time,refcursor) не существует, пропускается
DROP PROCEDURE
CREATE PROCEDURE
CREATE FUNCTION
psql:D:/BD/lr5/lr5.sql:159: ЗАМЕЧАНИЕ:  триггер "trg_lesson_no_conflicts" для отношения "lr3.lesson" не существует, пропускается
DROP TRIGGER
CREATE TRIGGER
CREATE FUNCTION
psql:D:/BD/lr5/lr5.sql:183: ЗАМЕЧАНИЕ:  триггер "trg_lesson_teacher_can_teach" для отношения "lr3.lesson" не существует, пропускается
DROP TRIGGER
CREATE TRIGGER
CREATE FUNCTION
psql:D:/BD/lr5/lr5.sql:217: ЗАМЕЧАНИЕ:  триггер "trg_lesson_discipline_in_program" для отношения "lr3.lesson" не существует, пропускается
DROP TRIGGER
CREATE TRIGGER
CREATE FUNCTION
psql:D:/BD/lr5/lr5.sql:251: ЗАМЕЧАНИЕ:  триггер "trg_lesson_room_capacity_ok" для отношения "lr3.lesson" не существует, пропускается
DROP TRIGGER
CREATE TRIGGER
CREATE FUNCTION
psql:D:/BD/lr5/lr5.sql:286: ЗАМЕЧАНИЕ:  триггер "trg_attestation_check_program_discipline" для отношения "lr3.attestation" не существует, пропускается
DROP TRIGGER
CREATE TRIGGER
CREATE FUNCTION
psql:D:/BD/lr5/lr5.sql:314: ЗАМЕЧАНИЕ:  триггер "trg_teacher_position_no_overlap" для отношения "lr3.teacher_position" не существует, пропускается
DROP TRIGGER
CREATE TRIGGER
CREATE FUNCTION
psql:D:/BD/lr5/lr5.sql:348: ЗАМЕЧАНИЕ:  триггер "trg_recalc_program_total_hours" для отношения "lr3.program_discipline" не существует, пропускается
DROP TRIGGER
CREATE TRIGGER
CREATE TABLE
CREATE FUNCTION
psql:D:/BD/lr5/lr5.sql:384: ЗАМЕЧАНИЕ:  триггер "trg_audit_lesson" для отношения "lr3.lesson" не существует, пропускается
DROP TRIGGER
CREATE TRIGGER
Courses=#

```

1. Реализация процедур согласно индивидуальному заданию, ч. 4:

Процедура № 1: Вывести расписание для заданной группы на указанный день недели.

SQL-команда теста процедуры № 1:

```

BEGIN;

CALL lr3.pr_get_group_schedule_by_dow(1, 1, 'c1');

FETCH ALL FROM c1;

COMMIT;

```

Результат выполнения теста процедуры № 1:

```

Courses=# BEGIN;
BEGIN
Courses=# CALL lr3.pr_get_group_schedule_by_dow(1, 1, 'c1');
  p_cur
-----
  c1
(1 строка)

Courses=# FETCH ALL FROM c1;
 lesson_date | lesson_time | lesson_type | discipline_name | room_number | room_type | last_name | first_name | middle_name
-----+-----+-----+-----+-----+-----+-----+-----+-----
 2026-03-02  | 09:00:00    | Лекция      | Базы данных    | 101          | Лекционная | Иванов   | Иван      | Иванович
 2026-03-02  | 11:30:00    | Практика    | Программирование | 205          | Компьютерный класс | Петрова  | Анна      | Сергеевна
 2026-03-09  | 09:00:00    | Лекция      | Базы данных    | 101          | Лекционная | Иванов   | Иван      | Иванович
 2026-03-09  | 11:30:00    | Практика    | Программирование | 205          | Компьютерный класс | Петрова  | Анна      | Сергеевна
 2026-03-16  | 09:00:00    | Лекция      | Базы данных    | 101          | Лекционная | Иванов   | Иван      | Иванович
 2026-03-16  | 11:30:00    | Практика    | Программирование | 205          | Компьютерный класс | Петрова  | Анна      | Сергеевна
 2026-03-23  | 09:00:00    | Лекция      | Базы данных    | 101          | Лекционная | Иванов   | Иван      | Иванович
 2026-03-23  | 11:30:00    | Практика    | Программирование | 205          | Компьютерный класс | Петрова  | Анна      | Сергеевна
 2026-04-20  | 11:30:00    | Практика    | Программирование | 205          | Компьютерный класс | Петрова  | Анна      | Сергеевна
(9 строк)

Courses=# COMMIT;
COMMIT
Courses=#

```

Процедура № 2: Запись (зачисление/перевод) слушателя в группу.

SQL-команда теста процедуры № 2:

```
SELECT trainee_id, group_id FROM lr3.trainee WHERE trainee_id = 1;  
CALL lr3.pr_enroll_trainee_to_group(1, 2);  
SELECT trainee_id, group_id FROM lr3.trainee WHERE trainee_id = 1;
```

Результат выполнения теста процедуры № 2:

```
Courses=# SELECT trainee_id, group_id FROM lr3.trainee WHERE trainee_id=1;  
trainee_id | group_id  
-----+-----  
          1 |          1  
(1 строка)  
  
Courses=# CALL lr3.pr_enroll_trainee_to_group(1, 2);  
CALL  
Courses=# SELECT trainee_id, group_id FROM lr3.trainee WHERE trainee_id=1;  
trainee_id | group_id  
-----+-----  
          1 |          2  
(1 строка)
```

Процедура № 3: Получение списка свободных аудиторий типа «Лекционная» на заданный день недели и время.

SQL-команда теста процедуры № 3:

```
BEGIN;  
CALL lr3.pr_free_lecture_rooms_by_dow(1, '10:00'::time, 'c2');  
FETCH ALL FROM c2;  
COMMIT;
```

Результат выполнения теста процедуры № 3:

```
Courses=# CALL lr3.pr_free_lecture_rooms_by_dow(1, '10:00'::time, 'c2');
p_cur
-----
c2
(1 строка)
```

```
Courses=# FETCH ALL FROM c2;
 room_id | room_number | capacity
-----+-----+-----
      1 |      101    |        90
      3 |      501    |       100
(2 строки)
```

```
Courses=# COMMIT;
COMMIT
```

```
Courses=# SELECT r.room_id, r.room_number, rt.type_name
Courses=# FROM lr3.room r
Courses=# JOIN lr3.room_type rt ON rt.room_type_id = r.room_type_id
Courses=# WHERE rt.type_name = 'Лекционная'
Courses=# ORDER BY r.room_id;
 room_id | room_number | type_name
-----+-----+-----
      1 |      101    | Лекционная
      3 |      501    | Лекционная
(2 строки)
```

```
Courses=# SELECT l.lesson_id, l.lesson_date, l.lesson_time, l.room_id
Courses=# FROM lr3.lesson l
Courses=# WHERE EXTRACT(ISODOW FROM l.lesson_date)::int = 1
Courses=# AND l.lesson_time = '10:00'::time
Courses=# ORDER BY l.lesson_date;
 lesson_id | lesson_date | lesson_time | room_id
-----+-----+-----+-----
(0 строк)
```

2. Реализация триггеров (вариант 2.1):

Триггер № 1: Запрещает ситуацию, когда на одну дату/время:

- одна и та же аудитория занята двумя занятиями;
- один преподаватель назначен на два занятия;
- одна группа имеет два занятия.

SQL-команда теста триггера № 1:

```
SELECT lesson_id, lesson_date, lesson_time, room_id, teacher_id, group_id
FROM lr3.lesson
```

```

ORDER BY lesson_id
LIMIT 10;

BEGIN;
UPDATE lr3.lesson
SET lesson_date = (SELECT lesson_date FROM lr3.lesson WHERE lesson_id = 1),
    lesson_time = (SELECT lesson_time FROM lr3.lesson WHERE lesson_id = 1),
    room_id      = (SELECT room_id      FROM lr3.lesson WHERE lesson_id = 1)
WHERE lesson_id = 2;

ROLLBACK;

```

Результат выполнения теста триггера № 1:

```

Courses=# SELECT lesson_id, lesson_date, lesson_time, room_id, teacher_id, group_id
Courses=# FROM lr3.lesson
Courses=# ORDER BY lesson_id
Courses=# LIMIT 10;
 lesson_id | lesson_date | lesson_time | room_id | teacher_id | group_id
-----+-----+-----+-----+-----+-----
         1 | 2026-04-15 | 09:05:00   |        1 |          1 |        1
         2 | 2026-04-20 | 11:30:00   |        2 |          2 |        1
         3 | 2026-04-06 | 09:00:00   |        1 |          1 |        3
         4 | 2026-04-05 | 14:00:00   |        1 |          1 |        1
        36 | 2026-02-15 | 09:00:00   |        1 |          1 |        1
        37 | 2026-02-11 | 11:30:00   |        2 |          2 |        1
        38 | 2026-02-19 | 09:00:00   |        1 |          1 |        1
        39 | 2026-02-12 | 13:00:00   |        2 |          2 |        1
        40 | 2026-02-21 | 11:30:00   |        1 |          1 |        1
        41 | 2026-02-13 | 09:00:00   |        2 |          2 |        1
(10 строк)

```

```

Courses=# BEGIN;
BEGIN
Courses=#
Courses=# UPDATE lr3.lesson
Courses=# SET lesson_date = (SELECT lesson_date FROM lr3.lesson WHERE lesson_id = 1),
Courses=# lesson_time = (SELECT lesson_time FROM lr3.lesson WHERE lesson_id = 1),
Courses=# room_id      = (SELECT room_id      FROM lr3.lesson WHERE lesson_id = 1)
Courses=# WHERE lesson_id = 2;
ОШИБКА: Аудитория занята: room_id=1 на 2026-04-15 09:05:00
КОНТЕКСТ: функция PL/pgSQL fn_lesson_no_conflicts(), строка 10, оператор RAISE
Courses=!#
Courses=!# ROLLBACK;
ROLLBACK
Courses=#

```

Триггер № 2: Проверяет наличие записи в discipline_teacher(teacher_id, discipline_id).

SQL-команда теста триггера № 2:


```

SELECT lesson_id, lesson_date, lesson_time, group_id, discipline_id,
teacher_id, room_id, lesson_type_id
FROM lr3.lesson
ORDER BY lesson_id
LIMIT 1;

SELECT t.teacher_id
FROM lr3.teacher t
WHERE NOT EXISTS (
    SELECT 1
    FROM lr3.discipline_teacher dt
    WHERE dt.teacher_id = t.teacher_id
        AND dt.discipline_id = 1
)
LIMIT 1;

INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id, discipline_id,
teacher_id, room_id, lesson_type_id)
VALUES ('2026-04-15', '10:00'::time, 1, 1, 2, 1, 1);

```

Результат выполнения теста триггера № 2:

```

Courses=# SELECT lesson_id, lesson_date, lesson_time, group_id, discipline_id, teacher_id, room_id, lesson_type_id
Courses=# FROM lr3.lesson
Courses=# ORDER BY lesson_id
Courses=# LIMIT 1;
 lesson_id | lesson_date | lesson_time | group_id | discipline_id | teacher_id | room_id | lesson_type_id
-----+-----+-----+-----+-----+-----+-----+-----
          1 | 2026-04-15 | 09:05:00   |         1 |              1 |          1 |         1 |              1
(1 строка)

Courses=# SELECT t.teacher_id
Courses=# FROM lr3.teacher t
Courses=# WHERE NOT EXISTS (
Courses=#     SELECT 1
Courses=#     FROM lr3.discipline_teacher dt
Courses=#     WHERE dt.teacher_id = t.teacher_id
Courses=#         AND dt.discipline_id = 1
Courses=# )
Courses=# LIMIT 1;
 teacher_id
-----
          2
(1 строка)

Courses=# INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id, discipline_id, teacher_id, room_id, lesson_type_id)
Courses=# VALUES ('2026-04-15', '10:00'::time, 1, 1, 2, 1, 1);
ОШИБКА: Преподаватель 2 не может вести дисциплину 1 (нет записи discipline_teacher)
КОНТЕКСТ: функция PL/pgSQL fn_lesson_teacher_can_teach(), строка 9, оператор RAISE
Courses=#

```

Триггер № 3: Дисциплина занятия обязана входить в программу группы через program_discipline.

SQL-команда теста триггера № 3:

```
SELECT l.lesson_id, l.group_id, tg.program_id, l.teacher_id, l.room_id,
l.lesson_type_id, l.lesson_date, l.lesson_time
FROM lr3.lesson l
JOIN lr3.training_group tg ON tg.group_id = l.group_id
ORDER BY l.lesson_id
LIMIT 1;

SELECT d.discipline_id
FROM lr3.discipline d
WHERE NOT EXISTS (
    SELECT 1
    FROM lr3.program_discipline pd
    WHERE pd.program_id = 1
        AND pd.discipline_id = d.discipline_id
)
LIMIT 1;

SELECT 1
FROM lr3.discipline_teacher
WHERE teacher_id = 1
    AND discipline_id = 3;

INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id, discipline_id,
teacher_id, room_id, lesson_type_id)
VALUES ('2026-04-15', '12:00'::time, 1, 3, 1, 1, 1);
```

Результат выполнения теста триггера № 3:


```

Courses=# FROM lr3.lesson l
Courses=# JOIN lr3.training_group tg ON tg.group_id = l.group_id
Courses=# ORDER BY l.lesson_id
Courses=# LIMIT 1;
 lesson_id | group_id | program_id | teacher_id | room_id | lesson_type_id | lesson_date | lesson_time
-----+-----+-----+-----+-----+-----+-----+-----
          1 |         1 |          1 |          1 |        1 |              1 | 2026-04-15 | 09:05:00
(1 строка)

Courses=# SELECT d.discipline_id
Courses=# FROM lr3.discipline d
Courses=# WHERE NOT EXISTS (
Courses=#   SELECT 1
Courses=#   FROM lr3.program_discipline pd
Courses=#   WHERE pd.program_id = 1
Courses=#   AND pd.discipline_id = d.discipline_id
Courses=# )
Courses=# LIMIT 1;
 discipline_id
-----
            3
(1 строка)

Courses=# SELECT 1
Courses=# FROM lr3.discipline_teacher
Courses=# WHERE teacher_id = 1
Courses=#   AND discipline_id = 3;
 ?column?
-----
         1
(1 строка)

Courses=# INSERT INTO lr3.lesson(lesson_date, lesson_time, group_id, discipline_id, teacher_id, room_id, lesson_type_id)
Courses=# VALUES ('2026-04-15', '12:00'::time, 1, 3, 1, 1, 1);
ОШИБКА: Дисциплина 3 не входит в программу 1 (группа 1)
КОНТЕКСТ: функция PL/pgSQL fn_lesson_discipline_in_program(), строка 19, оператор RAISE
Courses=#

```

Триггер № 4: Запрещает назначить аудиторию, если room.capacity < количество слушателей в trainee (по group_id).

SQL-команда теста триггера № 4:

```

SELECT l.lesson_id, l.group_id, l.lesson_date, l.lesson_time
FROM lr3.lesson l
ORDER BY l.lesson_id
LIMIT 3;

SELECT COUNT(*) AS trainees_cnt
FROM lr3.trainee
WHERE group_id = 3;

SELECT r.room_id, r.room_number, r.capacity
FROM lr3.room r
WHERE r.capacity < 4
AND NOT EXISTS (
    SELECT 1
    FROM lr3.lesson l
    WHERE l.room_id = r.room_id
    AND l.lesson_date = '2026-04-06'::date

```

```

        AND l.lesson_time = '09:00'::time
    )
ORDER BY r.capacity
LIMIT 1;

UPDATE lr3.lesson
SET room_id = 5
WHERE lesson_id = 3;

```

Результат выполнения теста триггера № 4:

```

Courses=# SELECT l.lesson_id, l.group_id, l.lesson_date, l.lesson_time
Courses=# FROM lr3.lesson l
Courses=# ORDER BY l.lesson_id
Courses=# LIMIT 3;
 lesson_id | group_id | lesson_date | lesson_time
-----+-----+-----+-----
          1 |         1 | 2026-04-15 | 09:05:00
          2 |         1 | 2026-04-20 | 11:30:00
          3 |         3 | 2026-04-06 | 09:00:00
(3 строки)

Courses=# SELECT COUNT(*) AS trainees_cnt
Courses=# FROM lr3.trainee
Courses=# WHERE group_id = 3;
 trainees_cnt
-----
            4
(1 строка)

Courses=# SELECT r.room_id, r.room_number, r.capacity
Courses=# FROM lr3.room r
Courses=# WHERE r.capacity < 4
Courses=# AND NOT EXISTS (
Courses=#     SELECT 1
Courses=#     FROM lr3.lesson l
Courses=#     WHERE l.room_id = r.room_id
Courses=#     AND l.lesson_date = '2026-04-06'::date
Courses=#     AND l.lesson_time = '09:00'::time
Courses=# )
Courses=# ORDER BY r.capacity
Courses=# LIMIT 1;
 room_id | room_number | capacity
-----+-----+-----
        5 |         503 |         2
(1 строка)

Courses=# UPDATE lr3.lesson
Courses=# SET room_id = 5
Courses=# WHERE lesson_id = 3;
ОШИБКА: В группе 3 4 чел., а вместимость аудитории 5 = 2
КОНТЕКСТ: функция PL/pgSQL fn_lesson_room_capacity_ok(), строка 19, оператор RAISE
Courses=#

```

Триггер № 5: Нельзя поставить аттестацию по дисциплине, которая не входит в программу слушателя.

SQL-команда теста триггера № 5:

```
SELECT tr.trainee_id, tg.program_id
FROM lr3.trainee tr
JOIN lr3.training_group tg ON tg.group_id = tr.group_id
ORDER BY tr.trainee_id
LIMIT 1;

SELECT d.discipline_id
FROM lr3.discipline d
WHERE NOT EXISTS (
    SELECT 1 FROM lr3.program_discipline pd
    WHERE pd.program_id = 2 AND pd.discipline_id = d.discipline_id
)
LIMIT 1;

SELECT attestation_format_id
FROM lr3.attestation_format
ORDER BY attestation_format_id
LIMIT 1;

INSERT INTO lr3.attestation(trainee_id, discipline_id, attestation_date,
result, attestation_format_id)
VALUES (1, 3, CURRENT_DATE, '5', 1);
```

Результат выполнения теста триггера № 5:

```

Courses=# SELECT tr.trainee_id, tg.program_id
Courses=# FROM lr3.trainee tr
Courses=# JOIN lr3.training_group tg ON tg.group_id = tr.group_id
Courses=# ORDER BY tr.trainee_id
Courses=# LIMIT 1;
   trainee_id | program_id
-----+-----
           1 |          2
(1 строка)

Courses=# SELECT d.discipline_id
Courses=# FROM lr3.discipline d
Courses=# WHERE NOT EXISTS (
Courses=#   SELECT 1 FROM lr3.program_discipline pd
Courses=#   WHERE pd.program_id = 2 AND pd.discipline_id = d.discipline_id
Courses=# )
Courses=# LIMIT 1;
   discipline_id
-----
              3
(1 строка)

Courses=# SELECT attestation_format_id
Courses=# FROM lr3.attestation_format
Courses=# ORDER BY attestation_format_id
Courses=# LIMIT 1;
   attestation_format_id
-----
                      1
(1 строка)

Courses=# INSERT INTO lr3.attestation(trainee_id, discipline_id, attestation_date, result, attestation_format_id)
Courses=# VALUES (1, 3, CURRENT_DATE, '5', 1);
ОШИБКА: Нельзя поставить аттестацию: дисциплина 3 не входит в программу 2 (слушатель 1)
КОНТЕКСТ:  функция PL/pgSQL fn_attestation_check_program_discipline(), строка 20, оператор RAISE
Courses=#

```

Триггер № 6: Запрещает пересекающиеся интервалы дат (start_date/end_date) по одному преподавателю.

SQL-команда теста триггера № 6:

```

SELECT teacher_position_id, teacher_id, job_position_id, start_date, end_date
FROM lr3.teacher_position
ORDER BY teacher_position_id
LIMIT 1;

INSERT INTO lr3.teacher_position(teacher_id, job_position_id, start_date,
end_date)
VALUES (
    1,
    1,
    '2015-09-01'::date + 1,
    COALESCE(CURRENT_DATE, '2015-09-01'::date + 10)
);

```

Результат выполнения теста триггера № 6:

```

Courses=# SELECT teacher_position_id, teacher_id, job_position_id, start_date, end_date
Courses=# FROM lr3.teacher_position
Courses=# ORDER BY teacher_position_id
Courses=# LIMIT 1;
teacher_position_id | teacher_id | job_position_id | start_date | end_date
-----+-----+-----+-----+-----
1 | 1 | 1 | 2015-09-01 |
(1 строка)

Courses=# INSERT INTO lr3.teacher_position(teacher_id, job_position_id, start_date, end_date)
Courses=# VALUES (
Courses=# 1,
Courses=# 1,
Courses=# '2015-09-01'::date + 1,
Courses=# COALESCE(CURRENT_DATE, '2015-09-01'::date + 10)
Courses=# );
ОШИБКА: Пересечение периодов должностей у teacher_id=1
КОНТЕКСТ: функция PL/pgSQL fn_teacher_position_no_overlap(), строка 14, оператор RAISE
Courses=#

```

Триггер № 7: Автоматически пересчитывает educational_program.total_hours как сумму часов дисциплин программы. Корректность подтверждается тем, что после изменения program_discipline поле total_hours изменилось без ручного UPDATE, и совпало с контрольной суммой SUM(...).

SQL-команда теста триггера № 7:

```

SELECT program_id, program_name, total_hours
FROM lr3.educational_program
ORDER BY program_id
LIMIT 1;

SELECT d.discipline_id
FROM lr3.discipline d
WHERE NOT EXISTS (
    SELECT 1 FROM lr3.program_discipline pd
    WHERE pd.program_id = 1 AND pd.discipline_id = d.discipline_id
)
LIMIT 1;

SELECT total_hours FROM lr3.educational_program WHERE program_id = 1;

SELECT SUM(d.lecture_hours + d.practice_hours + d.lab_hours) AS sum_hours
FROM lr3.program_discipline pd
JOIN lr3.discipline d ON d.discipline_id = pd.discipline_id
WHERE pd.program_id = 1;

```

```

BEGIN;

INSERT INTO lr3.program_discipline(program_id, discipline_id)
VALUES (1, 3);

SELECT total_hours FROM lr3.educational_program WHERE program_id = 1;

SELECT SUM(d.lecture_hours + d.practice_hours + d.lab_hours) AS sum_hours
FROM lr3.program_discipline pd
JOIN lr3.discipline d ON d.discipline_id = pd.discipline_id
WHERE pd.program_id = 1;

ROLLBACK;

```

Результат выполнения теста триггера № 7:

```

Courses=# SELECT program_id, program_name, total_hours
Courses=# FROM lr3.educational_program
Courses=# ORDER BY program_id
Courses=# LIMIT 1;
 program_id | program_name | total_hours
-----+-----+-----
          1 | Разработчик БД |          100
(1 строка)

Courses=# SELECT d.discipline_id
Courses=# FROM lr3.discipline d
Courses=# WHERE NOT EXISTS (
Courses=#   SELECT 1 FROM lr3.program_discipline pd
Courses=#   WHERE pd.program_id = 1 AND pd.discipline_id = d.discipline_id
Courses=# )
Courses=# LIMIT 1;
 discipline_id
-----
              3
(1 строка)

Courses=# SELECT total_hours FROM lr3.educational_program WHERE program_id = 1;
 total_hours
-----
          100
(1 строка)

Courses=#
Courses=# SELECT SUM(d.lecture_hours + d.practice_hours + d.lab_hours) AS sum_hours
Courses=# FROM lr3.program_discipline pd
Courses=# JOIN lr3.discipline d ON d.discipline_id = pd.discipline_id
Courses=# WHERE pd.program_id = 1;
 sum_hours
-----
          100
(1 строка)

```

```

Courses=# BEGIN;
BEGIN
Courses=#
Courses=# INSERT INTO lr3.program_discipline(program_id, discipline_id)
Courses-# VALUES (1, 3);
INSERT 0 1
Courses=#
Courses=# SELECT total_hours FROM lr3.educational_program WHERE program_id = 1;
total_hours
-----
150
(1 строка)

Courses=#
Courses=# SELECT SUM(d.lecture_hours + d.practice_hours + d.lab_hours) AS sum_hours
Courses-# FROM lr3.program_discipline pd
Courses-# JOIN lr3.discipline d ON d.discipline_id = pd.discipline_id
Courses-# WHERE pd.program_id = 1;
sum_hours
-----
150
(1 строка)

Courses=#
Courses=# ROLLBACK;
ROLLBACK
Courses=#

```

Дополнительно создала аудит для изменений расписания таблицы lesson.

SQL-команда теста аудита:

```

SELECT lesson_id, lesson_date, lesson_time FROM lr3.lesson ORDER BY lesson_id
LIMIT 5;

UPDATE lr3.lesson
SET lesson_time = lesson_time + interval '5 minute'
WHERE lesson_id = 1;

SELECT audit_id, changed_at, action, lesson_id
FROM lr3.lesson_audit
ORDER BY audit_id DESC
LIMIT 20;

```

Результат выполнения теста аудита:


```

Courses=# SELECT lesson_id, lesson_date, lesson_time FROM lr3.lesson ORDER BY lesson_id LIMIT 5;
 lesson_id | lesson_date | lesson_time
-----+-----+-----
          1 | 2026-04-15 | 09:00:00
          2 | 2026-04-20 | 11:30:00
          3 | 2026-04-06 | 09:00:00
          4 | 2026-04-05 | 14:00:00
         36 | 2026-02-15 | 09:00:00
(5 строк)

Courses=# UPDATE lr3.lesson
Courses=# SET lesson_time = lesson_time + interval '5 minute'
Courses=# WHERE lesson_id = 1;
UPDATE 1
Courses=# SELECT audit_id, changed_at, action, lesson_id
Courses=# FROM lr3.lesson_audit
Courses=# ORDER BY audit_id DESC
Courses=# LIMIT 20;
 audit_id |          changed_at          | action | lesson_id
-----+-----+-----+-----
          1 | 2026-05-12 03:02:35.405514 | UPDATE |          1
(1 строка)

```

Выводы:

В ходе лабораторной работы №5 были получены практические навыки разработки серверной логики в PostgreSQL с использованием процедур, функций и триггеров.

Для предметной области «Курсы» реализованы и протестированы 3 хранимые процедуры: получение расписания группы на заданный день недели, зачисление слушателя в группу и поиск свободных лекционных аудиторий на указанные день недели и время. Реализованы триггеры, обеспечивающие контроль целостности и бизнес-правил: предотвращение накладок в расписании, проверка прав преподавателя на ведение дисциплины, соответствие дисциплины программе группы, контроль вместимости аудитории, корректность аттестации по дисциплинам программы, отсутствие пересечения периодов должностей преподавателя, а также автоматический пересчёт суммарных часов программы при изменении состава дисциплин. Дополнительно реализован аудит изменений расписания с фиксацией операций INSERT/UPDATE/DELETE в журнале.

Тестирование показало корректность работы разработанных объектов: процедуры возвращают ожидаемые наборы данных, а триггеры предотвращают внесение некорректной информации и автоматически поддерживают согласованность связанных данных, что повышает надёжность и целостность базы данных.